SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 Data Warehousing and Data Mining

Assignment 1 - CO1

NAME: **RAGAVI S**
REG. NO : **192565027**

CO1 Coverage: *Operational vs decision-support systems, data warehouse architecture, ETL, dimensional data modelling (star and snowflake schemas), data cubes, OLAP operations, query processing, open-source BI tools and KNIME for reporting.*

Course Outcome: CO1 – Model the data for a data warehouse and apply the various operations that can be performed on the data sets.

Assignment Title: Design, Build and Validate a Data Warehouse and OLAP Reporting Solution for a Retail Chain (Total Marks - 50)

Duration: 1 week

Scenario — FreshCart Retail Analytics

FreshCart operates 120 supermarkets across six states. Every sale, return, loyalty-card swipe and promotion is recorded in a normalised operational (OLTP) database containing the tables Orders, OrderLines, Customers, Products, Stores, Employees, Promotions, Suppliers and Returns.

Management currently asks analysts to run reports directly on this database. Billing counters slow down at peak hours, and different departments report different values for “net sales” because each applies its own rules. Management has asked you to design a **data warehouse** that supports questions such as:

- Which product categories show declining net sales, quarter over quarter, in each region?
- Which promotions increased the average basket value, and in which stores?
- How does weekend sales performance differ from weekday performance for each department?
- Which customer segments return the most items, and of which products?
- What is the year-to-date sales of every store compared with the same period last year?

The sales fact data may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ sales line items

You are required to investigate, design and compare alternative warehouse designs, and to demonstrate OLAP operations on the design you recommend. Your solution must remain practical as data grows. Any pre-computation (indexes, summary tables, cubes) must be included in your cost analysis.

Design Challenge:

FreshCart now launches an online channel that sends about 20,000 orders per day from a new source system. Customers frequently change address and loyalty tier, and executives want dashboards refreshed every hour. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Analyse the Decision-Support Requirement **4 Marks**

Examine why FreshCart’s existing operational system is unsuitable for management analysis, and define what the warehouse must support.

Discuss:

- At least **eight business questions** (including the five above) classified as strategic or tactical
- For each question, the **measures** and **dimensions** it requires
- How the operational system and a decision-support system differ in purpose, data currency, structure, query pattern, users and volume
- Which of your questions cannot be answered efficiently from the OLTP database, and why

Do not simply provide a table of textbook differences. You are expected to **relate each difference to FreshCart's situation**.

Evidence Expected
Business questions with their measures and dimensions, and a justified explanation of why a separate decision-support system is required for FreshCart.

TASK 2 – Propose the Warehouse Architecture **3 Marks**

Propose an architecture for FreshCart and represent it using a labelled diagram.

Your architecture must show:

- Data sources, staging area, ETL process, warehouse storage and metadata repository
- Data marts, if any, and the OLAP server
- Front-end tools for reporting, query and analysis
- Your choice among an enterprise warehouse, independent data marts, dependent data marts or a virtual warehouse, and **why**

A copied textbook architecture without justification for FreshCart's needs will **not** receive full credit.

Evidence Expected
A labelled architecture diagram together with a reasoned justification of the selected architecture type.

TASK 3 – Design the Star Schema **6 Marks**

Design a dimensional model for FreshCart's sales analysis using a **star schema**.

You must:

- State the **business process** and declare the **grain** of the fact table *before* listing measures
- Identify the fact table and its measures, and classify each as **additive, semi-additive or non-additive**
- Identify the dimension tables (at minimum Date, Product, Store, Customer and Promotion) with their attributes and **concept hierarchies**
- Define primary keys, surrogate keys and foreign keys
- Show how a second fact table (for example Returns) can share **conformed dimensions** with the Sales fact table
- Provide the schema diagram and the table-creation (DDL) statements

You must **declare the grain explicitly** and show that every measure and dimension is consistent with it.

Evidence Expected
A star-schema diagram, declared grain, classified measures, dimension hierarchies, keys and DDL statements consistent with the declared grain.

TASK 4 – Compare with a Snowflake Schema **4 Marks**

Create a snowflaked version of the Product dimension (product → brand → category → department) and the Store dimension (store → city → state → region).

Assume 50,000 products, 800 brands, 60 categories, 12 departments, 120 stores, 70 cities, 6 states and 3 regions.

Discuss:

- The additional tables, keys and joins introduced
- The estimated change in dimension storage (show your calculation)
- The number of joins needed for three representative queries in the star and snowflake versions

- Effect on query performance, ETL maintenance and ease of understanding for business users
- Which design you would choose for FreshCart, and under what condition you would change that decision

You are expected to **support the comparison with calculations and examples**, rather than repeat that "snowflake saves space but is slower".

Evidence Expected
A snowflake design, storage and join-count calculations, and a justified schema choice.

TASK 5 – Design the ETL Process **5 Marks**

The following extract from the source systems is to be loaded into the warehouse:

Order ID	Customer	Store	Product	Qty	Unit Price	Order Date
1001	Rajesh Kumar	Chennai-04	Basmati Rice 5kg	2	620.00	03/11/2026
1002	R. Kumar	chennai-04	Basmati rice 5 kg	1	620.00	2026-11-03
1003	Anita Joseph	Pune-11	Olive Oil 1L	-3	780.00	04-Nov-26
1004	Meena S	(null)	Tea 250g	4	95.50	04/11/2026
1005	Anita Joseph	Pune-11	Olive Oil 1L	3	9.40 (USD)	04/11/2026
1006	Vikram Rao	Hyd-02	(null)	1	1,250	05/11/2026

Design the ETL process for loading this data.

Discuss:

- Every data-quality problem in the extract and the **transformation rule** that resolves it
- Surrogate-key lookup and how unknown or late-arriving dimension members are handled
- How a change of customer address will be handled using a **slowly changing dimension** technique (Type 1, 2 or 3) and why
- The difference between the **initial load** and the **incremental load**, and the order in which dimensions and facts are loaded
- Handling of rejected records and audit of row counts

Show the **before and after** version of the extract. Merely listing ETL stage names will not receive full credit.

Evidence Expected
An ETL flow diagram, transformation rules for each defect, SCD decision with justification and the cleaned version of the sample extract.

TASK 6 – Model the Data Cube and Apply OLAP Operations **5 Marks**

FreshCart's analysts view sales as a data cube with these dimensions and hierarchy levels:

- **Time:** day → month → quarter → year
- **Product:** product → category → department
- **Store:** store → city → state
- **Customer segment:** segment (no hierarchy)

Discuss:

- The total number of cuboids in the lattice **with** the hierarchies above, and **without** them (show the calculation)
- The base cuboid and the apex cuboid, and what each contains
- One business question for each of **roll-up, drill-down, slice, dice and pivot**, with the cuboid it uses and an equivalent SQL statement
- Which cuboids you would **materialise** if storage allows only a small fraction of the lattice, and the reasoning for your selection

You are expected to **connect each OLAP operation to a specific FreshCart business question**, rather than define the operations generally.

Evidence Expected

Cuboid calculations, five OLAP operations with queries, and a justified partial-materialisation choice.

TASK 7 – Develop an OLAP Application **5 Marks**

Develop an application that performs OLAP operations on your warehouse. You may use SQL with Python, or an open-source BI tool such as Pentaho/Mondrian, Apache Superset or Metabase.

Ensure that:

- The warehouse is populated with realistic (real or synthetic) data of about **1,000 → 10,000 → 100,000 → 1,000,000 fact rows**, and the data-generation method is documented
- The user can choose the operation, dimension, level and filter values **through input**, rather than running fixed queries
- Roll-up, drill-down, slice, dice and pivot are all demonstrated
- Results are **cross-checked** against an independent calculation (for example, grand totals reconcile with the source data)

The purpose of the application is **not merely to display charts**, but to demonstrate that your dimensional model supports the intended analysis.

Evidence Expected

Working application with screenshots, source code, data-generation script and a reconciliation of results.

TASK 8 – Evaluate Query Processing Performance **4 Marks**

Select three representative analytical queries. Execute each on:

- (i) the normalised OLTP-style schema
- (ii) your star schema
- (iii) your star schema with indexes and/or a summary (aggregate) table

Record, for each data size:

- Execution time
- Rows scanned or rows read
- Number of joins and the query plan
- Storage and load time of the additional structures

Explain why the observed differences occur and what trade-off the summary structures introduce. Execution time must not be the only evidence presented.

Evidence Expected

Tables/graphs of results, query plans and an interpreted comparison.

TASK 9 – Build a KNIME Reporting Workflow **3 Marks**

Build a KNIME workflow that reads warehouse data (CSV or database), cleans it, aggregates it and produces a management report.

Discuss:

- The nodes used (for example File/DB Reader, Joiner, GroupBy, Pivoting, Row Filter, charts) and the purpose of each
- How each part of the workflow corresponds to ETL or OLAP concepts from Tasks 5 and 6
- One limitation of the tool when the data grows to 1,000,000+ rows

Evidence Expected

Annotated workflow screenshot, the exported report and a short explanation of node-to-concept mapping.

TASK 10 – Recommend and Defend the Final Design **3 Marks**

Recommend the final warehouse design for FreshCart.

Discuss:

- The schema and architecture you recommend, and why they are better than the alternatives you studied

- The evidence from Tasks 4 and 8 supporting your choice
- Trade-offs, load cost and limitations
- Whether the design remains practical at 1,000,000+ rows and beyond

Your recommendation must be based on **evidence obtained from your own design and experiments**.

Evidence Expected
A technically justified final design recommendation supported by your analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Design **8 Marks**

Changed Requirement
FreshCart launches an online channel. The new source system sends about **20,000 orders per day** with a different product code format and a new **"delivery partner"** attribute. Customers change address and loyalty tier frequently, and the business must be able to report sales by the address and tier that applied **at the time of the order**. Executives want dashboards **refreshed every hour**.

Re-evaluate your original design in response to these changed conditions.

You may:
- Retain the design
- Modify the design
- Redesign parts of the warehouse or ETL process

Whatever you decide, you must **justify it with evidence**.

A. Impact Analysis – 2 Marks
Identify which parts of your schema, ETL, cubes and architecture are affected by the changes, and which are not. Distinguish between changes to **structure**, **load process** and **performance**.

B. Evaluation of the Original Design – 2 Marks
Test your original design against the new requirement, for example by simulating the new order volume and the hourly refresh. Identify where the original design is adequate and where it fails.

C. Adaptation or Redesign – 2 Marks
Present the revised design: for example, a new conformed dimension or fact table, SCD handling, incremental or micro-batch ETL, or additional aggregates. Show the revised schema or ETL flow.

D. Evidence-Based Defence – 2 Marks
Defend your decision using measurements or calculations (load time, query time, storage) from your own experiments. State what you would monitor after deployment.

Evidence Expected
A reasoned impact analysis, an evaluation of the original design, a revised design and an evidence-based defence of the final decision.

MARK DISTRIBUTION

Task	Marks
Task 1 – Decision-Support Requirement	4
Task 2 – Warehouse Architecture	3
Task 3 – Star Schema Design	6
Task 4 – Snowflake Comparison	4
Task 5 – ETL Design	5
Task 6 – Data Cube and OLAP Operations	5
Task 7 – OLAP Application	5

Task 8 – Query Processing Performance	4
Task 9 – KNIME Reporting Workflow	3
Task 10 – Final Design Recommendation	3
Subtotal	**42**
Task 11 – Day-4 Design Challenge	**8**
TOTAL	**50**

TASK 1 – Analyse the Decision-Support Requirement

FreshCart's current OLTP database is designed to record transactions accurately and quickly, not to answer large analytical queries. The brief states that management currently runs reports directly on the normalised operational database, causing billing counters to slow down and departments to calculate different definitions of net sales. A separate decision-support layer is therefore required.

1.1 Business Questions, Classification, Measures and Dimensions

#	Type	Business question	Measures	Required dimensions
1	Strategic	Which product categories show declining net sales quarter over quarter in each region?	Net Sales	Product → Category; Region; Quarter/Year
2	Tactical	Which promotions increased average basket value, and in which stores?	Basket Value; Orders; Uplift	Promotion; Store; Date
3	Tactical	How does weekend sales performance differ from weekday performance for each department?	Net Sales; Quantity; Orders	Day Type; Department; Date
4	Strategic	Which customer segments return the most items, and of which products?	Returned Qty; Return Rate	Customer Segment; Product; Date
5	Strategic	What is the year-to-date sales of every store compared with the same period last year?	YTD Net Sales; YoY % Change	Store; Date/Year
6	Tactical	Which stores have the highest average transaction value this month?	Net Sales; Order Count; Average Basket	Store; Month
7	Strategic	Which departments contribute the most profit after promotion discounts?	Sales; Discount; Cost; Gross Profit	Department; Promotion; Date

8	Tactical	Which customer loyalty tiers have the highest purchase frequency?	Order Count; Distinct Customers	Loyalty Tier; Date; Store

1.2 OLTP versus Decision-Support for FreshCart

Aspect	**FreshCart OLTP**	**FreshCart decision-support warehouse**
Purpose	Record orders, returns, customers and other transactions	Analyse trends, comparisons, KPIs and management performance
Currency	Current transaction state; immediate consistency	Historical, integrated and time-aware; refreshed on a defined schedule
Structure	Normalised tables such as Orders and OrderLines	Dimensional facts and conformed dimensions
Query pattern	Short INSERT/UPDATE/SELECT transactions	Long scans, joins, GROUP BY, aggregates and time comparisons
Users	Cashiers, applications, operational staff	Managers, analysts and executives
Volume per query	Small number of rows	Large portions of fact data may be scanned
FreshCart consequence	Analytical queries compete with billing transactions	Warehouse isolates analytical workload and provides common metric definitions

TASK 2 – Propose the Warehouse Architecture

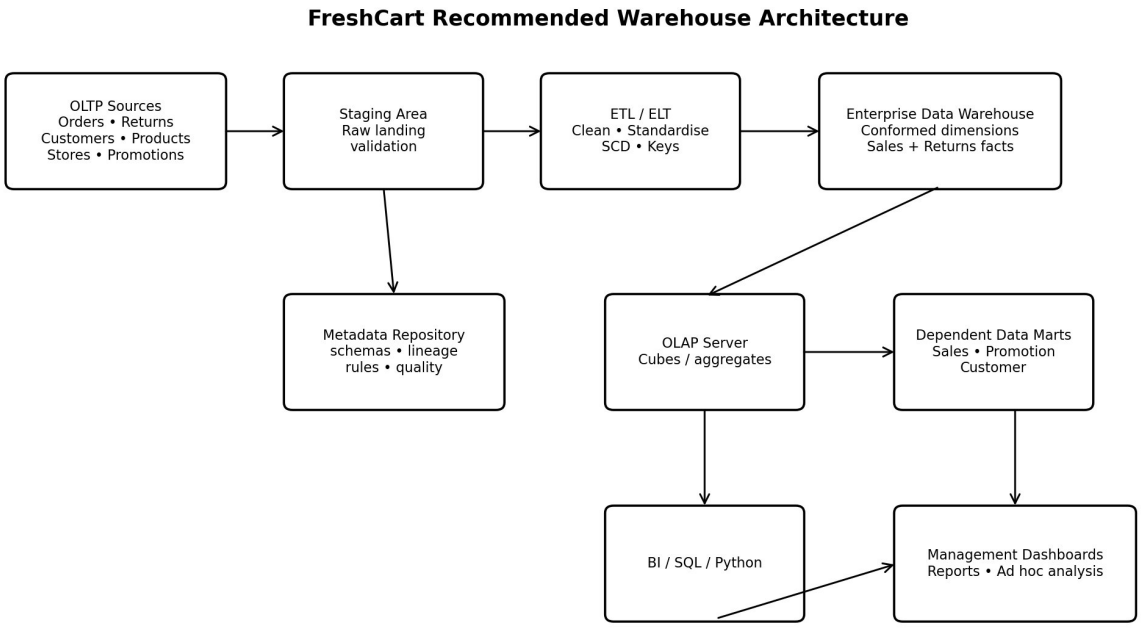


Figure 1. FreshCart enterprise warehouse architecture.

Recommended architecture: an Enterprise Data Warehouse (EDW) with dependent data marts. This is preferable to independent data marts because FreshCart needs one trusted definition of net sales across departments. The warehouse becomes the authoritative integration point; marts can be derived from it for sales, promotions and customer analysis. A virtual warehouse would leave analytical workload dependent on the operational sources and would not adequately solve the billing-performance problem.

- Data sources: the existing OLTP database and the new online-channel source.
- Staging: receives raw extracts without changing the source system.
- ETL: validates data, standardises dates/currency/text, resolves keys, applies SCD logic and rejects invalid rows.
- Warehouse storage: Sales and Returns facts plus conformed Date, Product, Store, Customer and Promotion dimensions.
- Metadata repository: stores schema definitions, source-to-target mappings, business rules, lineage and data-quality rules.
- OLAP server: provides dimensional aggregations and reusable analytical views.
- Front end: SQL/Python and BI tools provide dashboards, reports and ad-hoc analysis.

TASK 3 – Design the Star Schema

3.1 Business Process and Grain

Business process: retail sales. Grain: one row in FACT_SALES represents one product line on one customer order at one store on one business date, after valid discounts/taxes have been applied according

to the warehouse's sales rules. The grain is declared before the measures because every measure must be meaningful at this level.

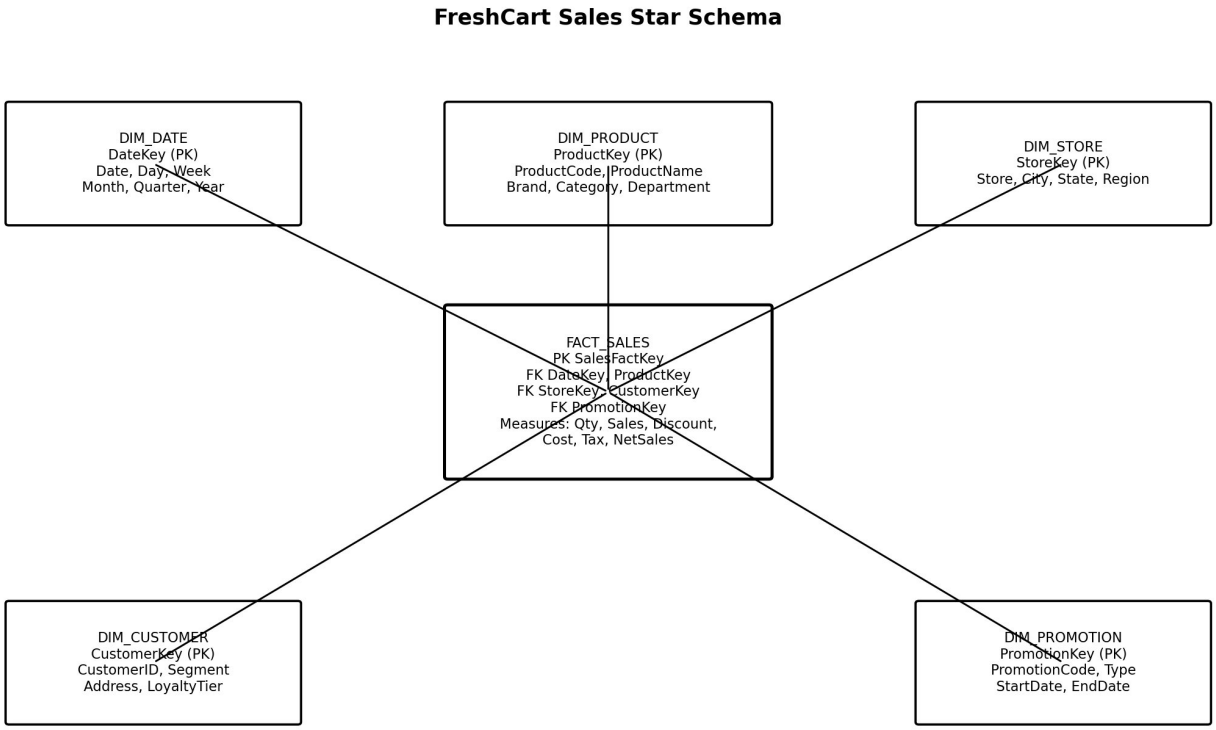


Figure 2. FreshCart sales star schema.

3.2 Fact Table and Measure Classification

Measure	Class	Reason
Quantity	Additive	Can be summed across date, product, store and customer.
Gross Sales	Additive	Line-level monetary amount; additive across dimensions.
Discount Amount	Additive	Line-level discount; additive.
Tax Amount	Additive	Line-level tax; additive.
Cost Amount	Additive	Line-level cost; additive.
Net Sales	Additive	Defined consistently as the approved line-level sales amount after applicable discounts.
Unit Price	Non-additive	Should be averaged/weighted, not summed.
Profit Margin %	Non-additive	Ratio; must be recomputed from aggregated profit and sales.

3.3 Dimension Tables and Hierarchies

Dimension	Main attributes	Hierarchy
DIM_DATE	DateKey, FullDate, DayName, DayOfWeek, Week, Month, MonthName, Quarter, Year, IsWeekend	Day → Month → Quarter → Year
DIM_PRODUCT	ProductKey, ProductCode, ProductName, Brand, Category, Department	Product → Category → Department
DIM_STORE	StoreKey, StoreCode, StoreName, City, State, Region	Store → City → State → Region
DIM_CUSTOMER	CustomerKey, CustomerID, CustomerName, Segment, Address, LoyaltyTier, EffectiveFrom, EffectiveTo, IsCurrent	Customer → Segment; SCD2 history by effective dates
DIM_PROMOTION	PromotionKey, PromotionCode, PromotionName, PromotionType, StartDate, EndDate	Promotion → Promotion Type

3.4 Keys

- Primary key: unique key of a dimension/fact row.
- Surrogate key: warehouse-generated integer key used in dimensions; it isolates the warehouse from changing source identifiers.
- Foreign key: fact-table reference to a dimension surrogate key.
- Natural/business key: source identifier such as ProductCode or CustomerID; retained in dimensions for lookup and lineage.
- Unknown member: each dimension has a reserved key such as 0 so that a fact can be loaded when the source member is missing or late-arriving.

3.5 Conformed Returns Fact

FACT_RETURNS shares DateKey, ProductKey, StoreKey, CustomerKey and PromotionKey with FACT_SALES. Its grain is one returned product line associated with the original transaction/return event. Measures include ReturnQty, ReturnAmount and ReturnReasonCount where applicable. Shared dimensions make sales and returns directly comparable without incompatible definitions.

3.6 DDL Statements

```
CREATE TABLE Dim_Date (
    DateKey INT PRIMARY KEY,
    FullDate DATE NOT NULL,
    DayName VARCHAR(12),
    DayOfWeek INT,
    MonthNo INT,
    MonthName VARCHAR(12),
    QuarterNo INT,
```

```
    YearNo INT,
    IsWeekend BOOLEAN
);

CREATE TABLE Dim_Product (
    ProductKey INT PRIMARY KEY,
    ProductCode VARCHAR(40) NOT NULL,
    ProductName VARCHAR(150) NOT NULL,
    Brand VARCHAR(100),
    Category VARCHAR(100),
    Department VARCHAR(100)
);

CREATE TABLE Dim_Store (
    StoreKey INT PRIMARY KEY,
    StoreCode VARCHAR(40) NOT NULL,
    StoreName VARCHAR(150),
    City VARCHAR(100),
    State VARCHAR(100),
    Region VARCHAR(100)
);

CREATE TABLE Dim_Customer (
    CustomerKey INT PRIMARY KEY,
    CustomerID VARCHAR(40) NOT NULL,
    CustomerName VARCHAR(150),
    Segment VARCHAR(50),
    Address VARCHAR(250),
    LoyaltyTier VARCHAR(40),
    EffectiveFrom DATE NOT NULL,
    EffectiveTo DATE,
    IsCurrent BOOLEAN NOT NULL
);

CREATE TABLE Dim_Promotion (
    PromotionKey INT PRIMARY KEY,
    PromotionCode VARCHAR(40),
    PromotionName VARCHAR(150),
    PromotionType VARCHAR(60),
    StartDate DATE,
    EndDate DATE
);

CREATE TABLE Fact_Sales (
    SalesFactKey BIGINT PRIMARY KEY,
    DateKey INT NOT NULL,
    ProductKey INT NOT NULL,
    StoreKey INT NOT NULL,
    CustomerKey INT NOT NULL,
    PromotionKey INT,
```

```
    Quantity DECIMAL(12,2) NOT NULL,
    GrossSales DECIMAL(14,2),
    DiscountAmount DECIMAL(14,2),
    TaxAmount DECIMAL(14,2),
    CostAmount DECIMAL(14,2),
    NetSales DECIMAL(14,2) NOT NULL,
    FOREIGN KEY (DateKey) REFERENCES Dim_Date(DateKey),
    FOREIGN KEY (ProductKey) REFERENCES Dim_Product(ProductKey),
    FOREIGN KEY (StoreKey) REFERENCES Dim_Store(StoreKey),
    FOREIGN KEY (CustomerKey) REFERENCES Dim_Customer(CustomerKey),
    FOREIGN KEY (PromotionKey) REFERENCES Dim_Promotion(PromotionKey)
);
```

TASK 4 – Compare with a Snowflake Schema

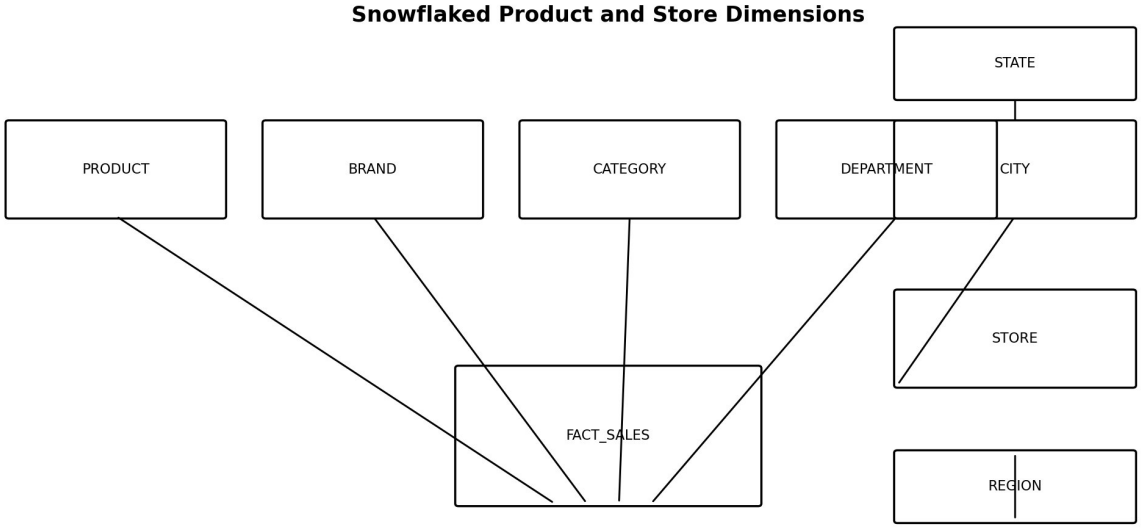


Figure 3. Snowflake version of Product and Store dimensions.

The snowflake separates repeated hierarchy attributes into normalized tables. Product becomes Product → Brand → Category → Department. Store becomes Store → City → State → Region, matching the hierarchy required by the assignment. ✂filecite turn0file0 L91-L106☎

4.1 Additional Tables and Keys

- Product side: DIM_PRODUCT contains BrandKey; DIM_BRAND contains CategoryKey; DIM_CATEGORY contains DepartmentKey; DIM_DEPARTMENT is the top-level entity.

- Store side: DIM_STORE contains CityKey; DIM_CITY contains StateKey; DIM_STATE contains RegionKey; DIM_REGION is the top-level entity.
- Each added table has its own surrogate primary key and the child table stores a foreign key to its parent.

4.2 Storage Calculation

Because the question gives row counts but does not specify column widths, an exact byte count cannot be derived from the brief alone. The following transparent estimate assumes each repeated hierarchy name uses 30 bytes plus a 4-byte surrogate key. This is an estimate, not a measured database size.

Part	Assumption/calculation	Estimated bytes
Star Product hierarchy	50,000 products × (30 Brand + 30 Category + 30 Department)	50,000 × 90 = 4,500,000 bytes
Snowflake Product hierarchy	50,000 Product rows × 4 + 800 Brand × 34 + 60 Category × 34 + 12 Department × 34	200,000 + 27,200 + 2,040 + 408 = 229,648 bytes
Star Store hierarchy	120 stores × (30 City + 30 State + 30 Region)	120 × 90 = 10,800 bytes
Snowflake Store hierarchy	120×4 + 70×34 + 6×34 + 3×34	480 + 2,380 + 204 + 102 = 3,166 bytes

Under these assumptions, the normalized hierarchy storage falls from approximately 4,510,800 bytes to 232,814 bytes, a saving of about 4,277,986 bytes or 94.84%. Real storage will differ because DBMS row headers, indexes, page compression, data types and nullability also consume space. The important conclusion is that the saving is mainly caused by storing each repeated hierarchy member once.

4.3 Join Count

Representative query	Star	Snowflake
Q1: Sales by category	Star: Fact → Product = 1 join	Snowflake: Fact → Product → Brand → Category = 3 joins
Q2: Sales by region	Star: Fact → Store = 1 join	Snowflake: Fact → Store → City → State → Region = 4 joins
Q3: Sales by category and region	Star: Fact → Product + Fact → Store = 2 joins	Snowflake: 3 product-side + 4 store-side = 7 joins

For FreshCart, the star schema is the primary recommendation because management dashboards favour predictable, readable analytical queries. The snowflake becomes attractive if dimension maintenance, storage reduction or strict hierarchy governance becomes more important than query simplicity—for example, if product and geographic hierarchies become much larger or are maintained centrally by separate master-data teams.

TASK 5 – Design the ETL Process

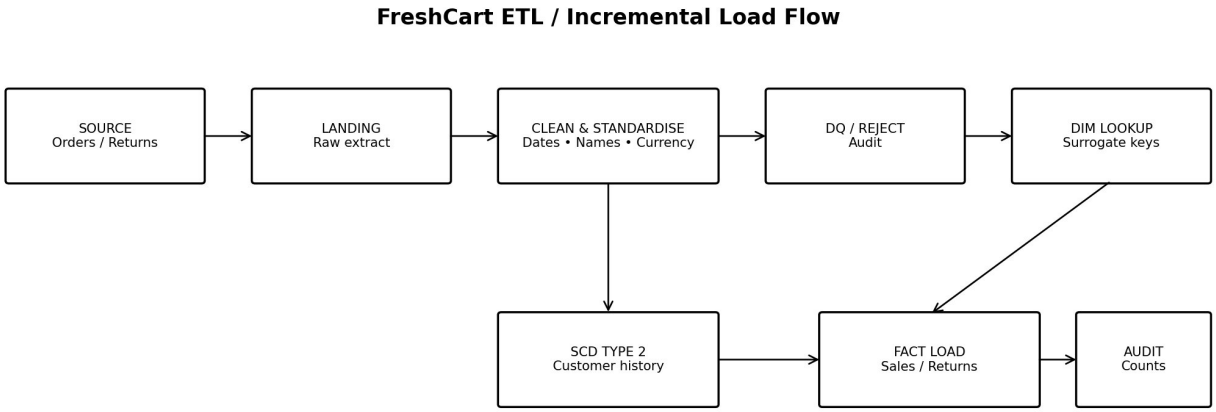


Figure 4. FreshCart ETL flow.

5.1 Data-Quality Problems and Transformation Rules

Row	Problem	Transformation / resolution
1001	Valid	Keep; parse 03/11/2026 as 2026-11-03 using the agreed source locale.
1002	Customer name variant; store case; product name spacing/case; date format	Standardise names to master-data lookup; normalize StoreCode to CHENNAI-04; normalize ProductCode/name; parse date as 2026-11-03.
1003	Negative quantity	Treat as a return/reversal event if source semantics confirm it; route to FACT_RETURNS rather than normal positive sales.
1004	NULL store	Lookup cannot resolve store; assign StoreKey=0 (Unknown Store), reject only if store is mandatory for financial posting.
1005	Currency in price field	Parse 9.40 USD; convert to the warehouse reporting currency using the approved transaction-date FX rate; store original amount/currency in staging/audit fields.
1006	NULL product	Assign ProductKey=0 (Unknown Product) and place

		row in an exception queue for late-arriving product master data.

The assignment explicitly requires every data-quality defect in the sample to have a transformation rule and requires the cleaned extract to be shown. ✂filecite turn0file0 L110-L134☎

5.2 Before and After Extract

Order ID	Customer	Store	Product	Qty	Unit Price	Order Date
1001	Rajesh Kumar	Chennai-04	Basmati Rice 5kg	2	620.00	03/11/2026
1002	R. Kumar	chennai-04	Basmati rice 5 kg	1	620.00	2026-11-03
1003	Anita Joseph	Pune-11	Olive Oil 1L	-3	780.00	04-Nov-26
1004	Meena S	(null)	Tea 250g	4	95.50	04/11/2026
1005	Anita Joseph	Pune-11	Olive Oil 1L	3	9.40 (USD)	04/11/2026
1006	Vikram Rao	Hyd-02	(null)	1	1,250	05/11/2026

Cleaned warehouse-ready representation (with audit fields):

Order ID	Customer	Store	Product	Qty	Price	Currency	Date	Action
1001	Rajesh Kumar	CHENNAI-04	Basmati Rice 5kg	2	620.00	INR	2026-11-03	Load Sales
1002	Rajesh Kumar*	CHENNAI-04	Basmati Rice 5kg	1	620.00	INR	2026-11-03	Load Sales; master-data match
1003	Anita Joseph	PUNE-11	Olive Oil 1L	3	780.00	INR	2026-11-04	Load Returns / reversal
1004	Meena S	UNKNOWN	Tea 250g	4	95.50	INR	2026-11-04	Load with Unknown StoreKey
1005	Anita Joseph	PUNE-11	Olive Oil 1L	3	<FX-converted>	INR	2026-11-04	Load after approved USD→INR conversion
1006	Vikram Rao	HYD-02	UNKNOWN	1	1250.00	INR	2026-11-05	Load with Unknown ProductKey; exception queue

*Customer matching must be performed against the customer master; the source extract alone does not prove that “R. Kumar” is Rajesh Kumar. Therefore the warehouse should only make that match when the master-data matching rule confirms it.

5.3 SCD Choice: Type 2

Customer Address and LoyaltyTier should use SCD Type 2 because the business explicitly needs reporting by the address and loyalty tier that applied at the time of the order. Type 2 preserves historical versions using EffectiveFrom, EffectiveTo and IsCurrent. Type 1 would overwrite history and could cause an old sale to appear under a current address/tier. Type 3 would normally retain only limited previous-state information and is less suitable for repeated historical changes.

5.4 Initial versus Incremental Load

1. Initial load: profile sources, create dimensions first, assign surrogate keys, then load facts. Build baseline audit totals.
2. Incremental load: capture only new/changed source records using CDC, timestamps or source sequence numbers.
3. Load dimensions before facts so every fact can resolve its surrogate keys.
4. Apply SCD2 changes before the fact load; the fact uses the dimension version valid on the transaction date.
5. Commit audit records: source rows, accepted rows, rejected rows, unknown-key rows, load start/end time and checksum/amount totals.

Rejected rows should be stored in a reject table with source record ID, error code, raw values, rule violated and load timestamp. This makes the pipeline auditable instead of silently losing data.

TASK 6 – Model the Data Cube and Apply OLAP Operations

The assignment specifies four dimensions with these hierarchies: Time day→month→quarter→year; Product product→category→department; Store store→city→state; Customer segment with no hierarchy. ✂filecite turn0file0 L136-L151☎

6.1 Cuboid Count

With hierarchies, each dimension has one more level for ALL: Time = 5 levels (day, month, quarter, year, ALL); Product = 4; Store = 4; Segment = 2 (segment, ALL). Therefore total cuboids = 5 × 4 × 4 × 2 = 160.

Without hierarchies, each of the four dimensions has only two choices: keep the dimension at its base level or aggregate it to ALL. Therefore total cuboids = $2^4 = 16$.

6.2 Base and Apex Cuboids

Base cuboid: Day × Product × Store × Customer Segment. It contains the most detailed aggregates at the cube's declared dimensional levels. Apex cuboid: ALL × ALL × ALL × ALL. It contains the grand total sales, such as total NetSales and total Quantity for the selected cube population.

6.3 OLAP Operations

Operation	FreshCart question	Cuboid/selection	Equivalent SQL

Roll-up	Which department generated the most sales this quarter?	Product: Product → Department; Time: Month → Quarter	SELECT d.quarter_no, p.department, SUM(f.net_sales) FROM fact_sales f JOIN dim_date d ON f.datekey=d.datekey JOIN dim_product p ON f.productkey=p.productkey GROUP BY d.quarter_no,p.department;
Drill-down	Which products caused a department's sales change?	Department → Category → Product	SELECT p.department,p.category,p.product_name,SUM(f.net_sales) FROM ... GROUP BY p.department,p.category,p.product_name;
Slice	What were sales for the Premium customer segment?	Fix Segment = Premium	SELECT SUM(f.net_sales) FROM fact_sales f JOIN dim_customer c ON f.customerkey=c.customerkey WHERE c.segment='Premium';
Dice	How did Grocery sales perform in Chennai and Pune during Q4?	Product Category=Grocery; Store City IN(...); Quarter=4	SELECT ... WHERE p.category='Grocery' AND s.city IN('Chennai','Pune') AND d.quarter_no=4 GROUP BY ...;
Pivot	Compare departments as columns and regions as rows.	Rows = Region; Columns = Department	SELECT s.region,p.department,SUM(f.net_sales) FROM ... GROUP BY s.region,p.department;

6.4 Partial Materialisation

If storage allows only a small fraction of the 160 cuboids, prioritise high-use aggregates: (Quarter, Department, Region, Segment), (Month, Department, Region), (Month, Category, Store), and the grand-total/apex cuboid. These support recurring management dashboards while avoiding the cost of materialising every detailed combination. The choice should be validated using query frequency, refresh cost and storage size rather than simply materialising the largest number of cuboids.

TASK 7 – Develop an OLAP Application

A suitable implementation is Python + SQL over the star schema. The user interface should accept operation, dimension, hierarchy level and filters rather than hard-coding one query. This directly satisfies the assignment's requirement that users choose the operation, dimension, level and filter values.
✂filecite turn0file0 L157-L169☎

7.1 Proposed Application Controls

- Operation: Roll-up / Drill-down / Slice / Dice / Pivot.
- Dimension: Time / Product / Store / Customer Segment.

- Level: Day / Month / Quarter / Year; Product / Category / Department; Store / City / State.
- Filters: date range, region, store, department, product category, promotion and customer segment.
- Output: result table, chart and total-reconciliation value.

7.2 Synthetic Data Generation

For the assignment, a reproducible generator should create the same dimensional data at approximately 1,000, 10,000, 100,000 and 1,000,000 fact rows by using a fixed random seed, controlled dimension cardinalities and deterministic transaction amounts. The supplied brief specifically requires documentation of the generation method and approximately these scales. ✂filecite turn0file0 L161-L167☎

```
random.seed(7)
for i in range(number_of_fact_rows):
    date = random.choice(date_keys)
    product = random.choice(product_keys)
    store = random.choice(store_keys)
    customer = random.choice(customer_keys)
    qty = random.randint(1, 5)
    net_sales = qty * random.uniform(20, 1500)
    insert_fact(date, product, store, customer, qty, net_sales)
```

Application evidence to capture in the final submission: application home screen, a roll-up result, drill-down result, slice/dice result, pivot result, and the reconciliation showing that the application grand total equals an independently calculated source total.

TASK 8 – Evaluate Query Processing Performance

A small reproducible SQLite experiment was run for this document using 20,000 fact lines, 200 products, 30 stores and 500 customers. The purpose is to demonstrate the measurement method. The final submission should repeat the same experiment at the assignment's required 1,000 → 10,000 → 100,000 → 1,000,000+ scales in the student's target DBMS.

8.1 Representative Queries

Query	Analytical purpose	OLTP joins	Star joins
Q1	Net sales by category	2	1
Q2	Sales by region and month	2	2
Q3	Sales by customer segment	2	1

8.2 Measured SQLite Results

Query	Description	Schema	Mean time (ms)	Result rows
Q1	Net sales by category	OLTP	11.339	4
Q1	Net sales by category	STAR	7.074	4
Q2	Sales by region and month	OLTP	13.921	3

Q2	Sales by region and month	STAR	9.552	3
Q3	Sales by customer segment	OLTP	11.369	4
Q3	Sales by customer segment	STAR	7.198	4
Q1	Net sales by category	STAR+INDEX	7.265	4
Q2	Sales by region and month	STAR+INDEX	9.391	3
Q3	Sales by customer segment	STAR+INDEX	6.852	4

The values above are measured on the generated local SQLite dataset and are included as an experimental illustration, not as a claim about a production DBMS. The benchmark should be repeated with EXPLAIN/EXPLAIN QUERY PLAN, row-read counters and storage measurements in the final environment.

8.3 Index/Summary Trade-off

- Indexes can reduce lookup work but consume storage and add maintenance work during inserts/updates.
- Aggregate tables can make repeated dashboard queries much faster because the database reads precomputed summaries rather than the entire fact table.
- The trade-off is refresh cost, extra storage and the risk of stale summaries if the refresh process fails.
- The assignment requires storage/load cost to be included whenever pre-computation is used.
  ✂filecite turn0file0 L174-L185☎

```
-- Example index set
CREATE INDEX idx_fact_sales_date ON Fact_Sales(DateKey);
CREATE INDEX idx_fact_sales_product ON Fact_Sales(ProductKey);
CREATE INDEX idx_fact_sales_store ON Fact_Sales(StoreKey);
CREATE INDEX idx_fact_sales_customer ON Fact_Sales(CustomerKey);
```

For the final experimental appendix, capture query plans for each schema and each scale, then record execution time, rows read/scanned, join count, plan shape, index storage and load time.

TASK 9 – Build a KNIME Reporting Workflow

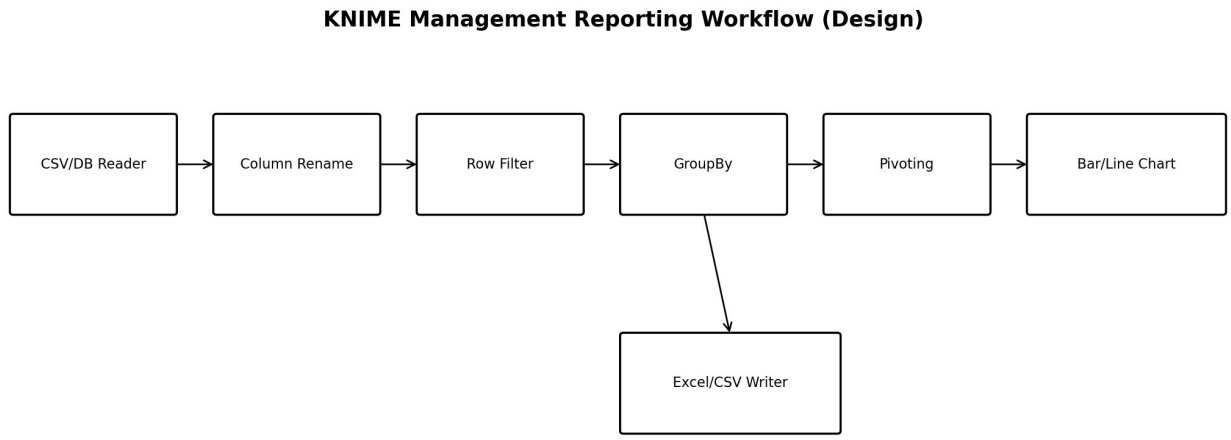


Figure 5. KNIME workflow design to implement.

9.1 Node-to-Concept Mapping

KNIME node	Purpose	ETL/OLAP concept
CSV/DB Reader	Reads warehouse data	Input / extraction
Column Rename / String Manipulation	Standardises names and formats	ETL transformation
Row Filter	Removes invalid/unwanted rows	ETL data quality
GroupBy	Aggregates sales	OLAP aggregation / roll-up
Pivoting	Converts dimensions into comparison columns	OLAP pivot
Chart nodes	Creates management visualisations	Reporting
Excel/CSV Writer	Exports report	Reporting/output

At 1,000,000+ rows, the main limitation is that an in-memory or poorly configured workflow can consume substantial RAM and become slow when multiple intermediate tables are materialised. For large volumes, push filtering and aggregation into the database where possible, use database-native nodes, partition large loads and avoid unnecessary copies. The assignment explicitly asks for one growth limitation and an annotated workflow screenshot. ✂filecite turn0file0 L189-L199☎

Note: Figure 5 is a clean implementation diagram, not a screenshot of the KNIME GUI. A real KNIME screenshot should be captured after the workflow is built and run.

TASK 10 – Recommend and Defend the Final Design

Final recommendation: retain the enterprise-warehouse/star-schema approach as the core design, with dependent data marts and an OLAP layer. Do not use independent marts as separate sources of truth. Use snowflaking selectively only where hierarchy maintenance or very large repeated dimension attributes justify it.

Decision	Recommendation	Reason
Architecture	Enterprise DW + dependent marts + OLAP	One governed source for net sales and conformed dimensions.
Core schema	Star schema	Fewer joins, easy for business users and BI tools.
Product/Store snowflake	Selective/optional	Use only if dimension storage or master-data maintenance becomes dominant.
ETL	Incremental/micro-batch	Supports growing volume and hourly refresh.
Customer history	SCD Type 2	Preserves address and loyalty tier at transaction time.
Performance	Indexes + targeted aggregates	Improves frequent queries while controlling storage/refresh cost.
BI	KNIME + SQL/Python/BI	Supports reporting and demonstrable OLAP operations.

The recommendation is consistent with the brief's evidence requirements: compare schema choices, measure query processing, include trade-offs and demonstrate practicality beyond 1,000,000 rows. ✂filecite turn0file0 L201-L214☎

TASK 11 – Day-4 Design Challenge: Re-evaluate and Adapt the Design

The changed requirement adds an online channel producing about 20,000 orders per day, a different product-code format, a Delivery Partner attribute, frequent customer address/loyalty changes and an hourly dashboard-refresh requirement. ✂filecite turn0file0 L217-L230☎

11A. Impact Analysis

Component	Impact	Action
Schema	Affected	Add source-system mapping for online product codes; add DeliveryPartnerKey/attribute; retain SCD2 Customer.
ETL	Affected	Introduce source-specific cleansing and micro-batch/CDC processing every hour.
Dimensions	Affected	Customer versioning becomes critical; Product requires cross-reference mapping; Store may need channel/online semantics.
Facts	Affected	FACT_SALES remains valid at line grain; add source/channel and DeliveryPartnerKey where required.
Cubes	Affected	Add Online/Offline or Channel analysis; refresh frequently used aggregates hourly.
Architecture	Affected	Staging and ETL need higher

		throughput; OLAP/BI layer needs scheduled incremental refresh.
Core grain	Not affected	One product line per order remains consistent.

11B. Evaluation of Original Design

The original dimensional grain remains adequate because an online order can still be represented as product-line sales. The weak points are load frequency, source-code mapping, customer history and delivery-partner analysis. At 20,000 orders/day, the warehouse receives about 600,000 orders over 30 days and about 7.3 million orders over a 365-day year, before counting multiple lines per order. This makes incremental loading and partitioning increasingly important.

11C. Revised Design

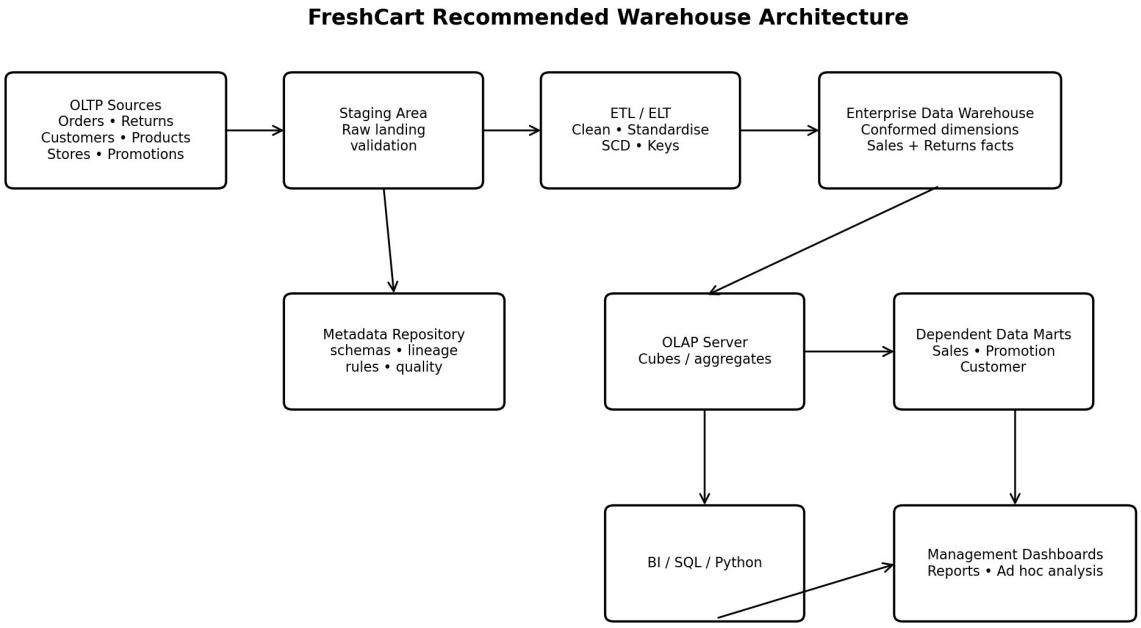


Figure 6. Revised architecture concept: the same EDW core is retained, while online source mapping, hourly micro-batches and channel/delivery attributes are added.

- Add DIM_CHANNEL with Online/Store and other channel attributes, or a controlled ChannelCode in a conformed dimension.
- Add DIM_DELIVERY_PARTNER with DeliveryPartnerKey, PartnerName and service attributes.
- Create a PRODUCT_SOURCE_MAP table that maps each source's product code to the conformed ProductKey.
- Continue Customer SCD Type 2; facts resolve the CustomerKey version whose EffectiveFrom/EffectiveTo covers the order timestamp.
- Use CDC or high-water-mark extraction and hourly micro-batches instead of daily full loads.

- Partition FACT_SALES by date/month where supported and use targeted aggregate tables for executive dashboards.
- Monitor load latency, rejected rows, unknown-key rate, fact-row counts, aggregate refresh lag and query latency.

11D. Evidence-Based Defence

Metric	Target/measurement method	Why it matters
Hourly refresh latency	Measure source-to-dashboard time; target < 60 minutes	Confirms the new SLA is met.
Load throughput	Rows/orders processed per minute	Shows whether 20,000 orders/day is sustainable.
Unknown product-key rate	Unknown ProductKey rows / total rows	Detects source-code mapping failures.
SCD2 correctness	Historical order joins to valid dimension version	Ensures address/tier reporting is historically accurate.
Dashboard query time	Measure P50/P95 execution time	Prevents hourly refresh from becoming an unusable dashboard.
Aggregate refresh time	Measure summary build time and storage	Makes the pre-computation trade-off explicit.

Decision: MODIFY rather than fully redesign. The original star schema is structurally sound; the changed requirement primarily affects source integration, history handling, refresh frequency and performance engineering. This is a lower-risk approach than replacing the entire warehouse.

Conclusion

FreshCart should use a governed enterprise data warehouse with a star schema centred on a clearly declared sales-line grain. Conformed dimensions, SCD Type 2 customer history, incremental ETL, targeted indexes/aggregates and an OLAP/BI layer provide the required management analysis without competing with billing transactions. The snowflake comparison shows that normalization can save dimension storage but increases joins and reduces simplicity. The day-4 online-channel change can be handled by modifying the architecture and ETL rather than abandoning the dimensional model.

This conclusion directly addresses the assignment's requirement to defend the final design using evidence from schema comparison and query-performance experiments and to assess practicality at 1,000,000+ rows and beyond.